VeraGrid Implicit JIT Engine

Mathematical Foundations, Numerical Validation, and Software Architecture

VeraGrid Development Team
Computational Power Systems Group

January 28, 2026

Contents

1 Introduction **3**

2 Mathematical Validation Suite **4**
2.1 Discretization Accuracy 4
2.1.1 Test 01: Backward Euler Accuracy (L-Stable) 4
2.1.2 Test 02: Trapezoidal Rule Accuracy (A-Stable) 4
2.2 Order of Convergence Analysis 4
2.2.1 Test 04: Proof of $O(h^2)$ Convergence 4
2.3 Stability & Physics Tests 5
2.3.1 Test 05: L-Stability (Stiffness Handling) 5
2.3.2 Test 07: Energy Conservation (Symplectic Behavior) 5
2.4 Non-Linear & Control Tests 5
2.4.1 Test 06: Non-Linear Jacobian Consistency 5
2.4.2 Test 03 & 08: Boundary & Time Injection 6
2.5 Index-1 DAEs, Trapezoidal Method Order, and Event Handling in EMT 6
2.5.1 Problem class: DAEs in EMT and the concept of index 6
2.5.2 Order of the trapezoidal method and conditions of validity 7
2.5.3 Events, discontinuities, and order degradation in EMT 7
2.5.4 Practical conclusion for the implicit engine 9

3 Functional Demos: EMT Capabilities **10**
3.1 Buck/Boost Converter (The "Stress Test") 10
3.2 Method Comparison: Damping vs. Accuracy 10

4 Transmission Line Architecture (Hybrid Solver) **11**
4.1 Theoretical Basis: Bergeron Method 11
4.2 Hybrid Architecture: Implicit Core + Boundary Injection 11

5 Scalability and Performance Limits **12**
5.1 Benchmark Configuration 12
5.2 Empirical Results 12
5.3 Analysis of Results 12
5.3.1 Runtime Speed ($O(N)$) 12
5.3.2 Compilation Bottleneck 13

6 Software Architecture & JIT Compiler **14**
6.1 JIT Compiler Innovations (`jit_compiler.py`) 14
6.1.1 Visitor Pattern for Discretization 14
6.1.2 Common Subexpression Elimination (CSE) 14
6.2 Block Solver Engine (`block_solver.py`) 14
6.2.1 Hybrid Jacobian Strategy 14
6.2.2 BDF2 Bootstrapping 14

6.3 Transition to a Monolithic Implicit DAE Solver . . . 15
6.3.1 Distributed Implicit Formulation . . . 15
6.3.2 Numerical Advantages of the Monolithic Formulation . . . 15
6.3.3 Enabling Solver-Level Diagnostics and Traceability . . . 16

7 Advanced Topics: Scalability and Initialization 17
7.1 Steady-State Initialization Strategy . . . 17
7.1.1 Theoretical Formulation . . . 17
7.1.2 Solver Implementation . . . 17
7.2 Scalability Analysis (Sparse Matrices) . . . 17
7.2.1 Computational Complexity . . . 17
7.2.2 JIT Compiler Memory Efficiency . . . 18

8 Numerical Diagnostics and Solver Observability 19
8.1 Motivation for Solver Observability . . . 19
8.2 Newton–Raphson Diagnostic Framework . . . 19
8.2.1 Monitored Quantities During Newton Iterations . . . 20
8.2.2 Diagnostic Wrapper Around the Linear Solve . . . 20
8.3 Traceability and Numerical Analysis . . . 21
8.4 Scope, Assumptions, and Current Limitations . . . 21

9 User Guide: New Parameters, Diagnostics, and Usage 22
9.1 New Simulation Parameters . . . 22
9.2 Combining Implicit and Boundary injection . . . 23
9.3 Newton Diagnostics and Trace Collection . . . 23
9.3.1 Newton Trace Collector . . . 23
9.3.2 Enabling Diagnostics in a Simulation . . . 24
9.3.3 Post-Processing and Analysis . . . 24
9.4 Intended Usage and Scope . . . 25

10 Implementation Roadmap and Planned Solver Extensions 26
10.1 Roadmap Overview . . . 26
10.2 Stage 1: Solver Robustness and Diagnostics . . . 27
10.3 Stage 2: Event Handling and Order Preservation . . . 27
10.4 Stage 3: Initialisation and Consistency . . . 28
10.5 Stage 4: Validation and Testing . . . 28

Chapter 1

Introduction

This document serves as the definitive technical reference for the **VeraGrid Implicit JIT Solver**. It details the mathematical validation of the engine, the architectural innovations in Just-In-Time (JIT) compilation using Numba, and the hybrid methodologies used to solve complex Electromagnetic Transient (EMT) problems involving distributed parameters and stiff non-linearities.

The engine represents a shift from legacy interpretation-based solvers to a compiled, sparse-matrix optimized architecture capable of handling hybrid Differential-Algebraic Equations (DAEs) with discrete event handling.

Chapter 2

Mathematical Validation Suite

The integrity of the solver is verified via the `discretization_verify.py` suite. This chapter details the theoretical basis and the specific validation logic for each of the unit tests, confirming the engine's numerical properties.

2.1 Discretization Accuracy

2.1.1 Test 01: Backward Euler Accuracy (L-Stable)

Theory: Backward Euler is an implicit, first-order method ($O(h)$). It is L-Stable, making it ideal for stiff systems where high-frequency vibrations must be damped. For a linear decay equation $\dot{x} = -kx$, the difference equation is:

$$x_{n+1} - x_n = h(-kx_{n+1}) \implies x_{n+1} = \frac{x_n}{1 + hk} \tag{2.1}$$

Validation: The test compiles a kernel for $\dot{x} = -x$ ($k = 1$) with $h = 0.1$. It asserts that the JIT-compiled result matches the algebraic formula to 7 decimal places. This confirms the compiler correctly maps symbolic variables to the memory array indices for this scheme.

2.1.2 Test 02: Trapezoidal Rule Accuracy (A-Stable)

Theory: The Trapezoidal rule (Crank-Nicolson) is an implicit, second-order method ($O(h^2)$). It is A-Stable and Symplectic (energy-preserving for linear Hamiltonian systems). The discrete update map is:

$$x_{n+1} = x_n + \frac{h}{2}(-kx_n - kx_{n+1}) \implies x_{n+1} = x_n \left(\frac{1 - \frac{hk}{2}}{1 + \frac{hk}{2}} \right) \tag{2.2}$$

Validation: This test verifies that the symbolic visitor pattern in `jit_compiler.py` generates the correct code structure, specifically:

```
# Generated code structure
((2.0/h) * (states[i] - history[i]) - d_history[i])
```

2.2 Order of Convergence Analysis

2.2.1 Test 04: Proof of $O(h^2)$ Convergence

Objective: Prove mathematically that the solver achieves Order 2 convergence for the Trapezoidal method. **Methodology:** We solve $\dot{x} = -x, x(0) = 1$ using two step sizes: $h_1 = 0.1s$ and

$h_2 = 0.05s$. The Global Truncation Error (GTE) ratio is calculated as:

$$R = \frac{|x_{num}(h_1) - x_{exact}|}{|x_{num}(h_2) - x_{exact}|} \approx 2^p \tag{2.3}$$

For $p = 2$, R should approach 4.0.

Critical Implementation Detail: The Trapezoidal method relies on the history of the derivative ($d_history$).

- If initialized with $dx_0 = 0$ (default) when the physical derivative is $\dot{x}(0) = -1$, the first step introduces an error of $O(h)$. This pollutes the global error, reducing it to Order 1 ($R \approx 2.0$).
- **Fix:** The test explicitly passes `dx0=[-1.0]`. With consistent initialization, the test confirms $R > 3.8$, proving rigorous $O(h^2)$ behavior.

2.3 Stability & Physics Tests

2.3.1 Test 05: L-Stability (Stiffness Handling)

Theory: L-Stability requires that for $\dot{x} = \lambda x$, as $Re(\lambda)h \to -\infty$, the numerical solution $x_{n+1} \to 0$ rapidly (spectral radius $\rho(M) \to 0$). **Scenario:** $\dot{x} = -1000x$ (Time constant $\tau = 1ms$) simulated with a coarse step $h = 0.1s$ ($100 \times \tau$). **Result:**

- Explicit methods (Forward Euler): Stability region is $|1 + h\lambda| < 1$. Here, $|1 - 100| = 99$, leading to massive explosion.
- VeraGrid (BackEuler): $x_{n+1} \approx \frac{1}{101} x_n$. The solution decays monotonically to zero without oscillation.

This proves the engine can handle parasitic inductances/capacitances (snubbers) without requiring nanosecond time steps.

2.3.2 Test 07: Energy Conservation (Symplectic Behavior)

Theory: An undamped LC oscillator ($\ddot{x} + x = 0$) conserves Hamiltonian energy $H = \frac{1}{2}x^2 + \frac{1}{2}v^2$.

- **Backward Euler:** Introduces numerical viscosity. Energy decays exponentially ($x_{n+1} < x_n$).
- **Trapezoidal:** Is a symplectic integrator. It preserves the shadow Hamiltonian. Energy oscillates slightly but stays bounded indefinitely.

Validation: The test simulates for 20 seconds. BackEuler amplitude decays to < 0.5. Trapezoidal maintains amplitude > 0.99. This validates Trapezoidal for power system phasors and long-term transient stability studies.

2.4 Non-Linear & Control Tests

2.4.1 Test 06: Non-Linear Jacobian Consistency

Scenario: $\dot{x} = x^2$. This system has a finite-time singularity ($x(t) \to \infty$ as $t \to 1$) and a rapidly changing derivative Jacobian $J = 2x$. **Validation:** The solver uses a Newton-Raphson loop. Convergence implies that the Jacobian matrix J (calculated via Finite Differences in small systems) accurately represents the slope of the residual function $F(x)$. Convergence to the analytical solution validates the internal linear algebra loop.

2.4.2 Test 03 & 08: Boundary & Time Injection

Mechanism: These tests validate the `boundary_update_fn` and parameter mapping.

- **Test 03:** Changes a physical parameter k from 1.0 to 0.0 at runtime.
- **Test 08:** Injects simulation time t into a dummy event parameter to solve non-autonomous equations $\dot{x} = t$.

Architecture Note: The solver separates the internal `glob_time` (used for stepping) from the model parameters. To solve time-dependent equations, time must be injected as an explicit parameter to avoid Numba typing errors.

2.5 Index-1 DAEs, Trapezoidal Method Order, and Event Handling in EMT

2.5.1 Problem class: DAEs in EMT and the concept of index

EMT (Electromagnetic Transients) models are commonly formulated as a differential–algebraic equation (DAE) system of the form:

$$\begin{aligned} \dot{x}(t) &= f\big(x(t), y(t), p(t)\big), \\ 0 &= g\big(x(t), y(t), p(t)\big), \end{aligned} \tag{2.4}$$

where $x \in \mathbb{R}^{n_x}$ denotes differential variables (states), $y \in \mathbb{R}^{n_y}$ algebraic variables (typically nodal voltages, auxiliary branch currents, constraint multipliers, or internal device variables), and $p(t)$ collects parameters, including those updated by discrete events.

The *index* of a DAE can be understood operationally as the minimum number of time differentiations required on the algebraic constraint $g(x, y, p) = 0$ to obtain an explicit dynamic expression (i.e., to express $\dot{y}$ in a well-defined manner). For (2.4), the system is an **index-1** DAE if the algebraic Jacobian with respect to y is nonsingular in the operating region:

$$\det\left(\frac{\partial g}{\partial y}(x, y, p)\right) \neq 0. \tag{2.5}$$

Under condition (2.5), the algebraic equation implicitly defines a function $y = \varphi(x, p)$ (by the implicit function theorem), and the dynamics can be equivalently written as an ordinary differential equation:

$$\dot{x} = f\big(x, \varphi(x, p), p\big). \tag{2.6}$$

From a numerical perspective, (2.5) is precisely the requirement that allows the implicit coupling (x, y) to be robustly solved using Newton–Raphson methods, without the need for index reduction techniques.

Relevance to EMT. In EMT formulations based on Modified Nodal Analysis (MNA), algebraic equations typically arise from KCL/KVL constraints and constitutive relations that do not involve time derivatives of algebraic variables (e.g., nodal voltages). As a result, the resulting systems are generally index-1 under normal operating conditions. However, situations of *near singularity* or *effective index change* may arise in the presence of:

- ideal switching (branches transitioning between open-circuit and short-circuit with extreme conductances),
- degenerate topologies (floating nodes, islands without reference),

- additional non-regularised constraints (hard projections in limiters),
- saturation or limiting behaviour modelled via complementarity-type discontinuities.

Therefore, explicitly stating that the solver is designed for index-1 DAEs, and describing how this condition is preserved through practical modelling and regularisation, is an essential part of a "bulletproof" technical documentation.

2.5.2 Order of the trapezoidal method and conditions of validity

The trapezoidal method (also known as Crank–Nicolson in the ODE literature), applied to

$$\dot{x}(t) = f(x(t), t), \tag{2.7}$$

is defined as:

$$x_{n+1} = x_n + \frac{h}{2}\left(f(x_n, t_n) + f(x_{n+1}, t_{n+1})\right), \tag{2.8}$$

where $h = t_{n+1} - t_n$.

Why it is second order. Equation (2.8) corresponds to approximating the exact integral

$$x(t_{n+1}) - x(t_n) = \int_{t_n}^{t_{n+1}} f(x(t), t)\, dt \tag{2.9}$$

using the trapezoidal quadrature rule, whose local truncation error is $\mathcal{O}(h^3)$ provided that f is sufficiently smooth (at least C^2) over the integration interval. By standard consistency and zero-stability arguments, this results in a global error of $\mathcal{O}(h^2)$, i.e., second-order accuracy.

Derivative recurrence form and initial consistency. In implicit DAE-oriented implementations, the trapezoidal rule is often rewritten as a derivative recurrence:

$$\dot{x}_{n+1} \approx \frac{2}{h}(x_{n+1} - x_n) - \dot{x}_n, \tag{2.10}$$

which is algebraically equivalent to (2.8). However, (2.10) makes explicit a practical requirement: to observe second-order global accuracy, the initial condition must be *consistent*, namely

$$\dot{x}_0 = f(x_0, t_0) \quad \text{and (for DAEs)} \quad g(x_0, y_0, p_0) = 0. \tag{2.11}$$

If (2.11) is not satisfied (for instance, if $\dot{x}_0$ is arbitrarily initialised to zero), the first step introduces an $\mathcal{O}(h)$ defect that may dominate the global error, degrading the observed behaviour to first order despite the method being formally second order. This distinction is particularly relevant in EMT, where initial conditions are typically derived from RMS power flow solutions and/or steady-state conditions that require an additional mapping procedure to EMT states.

2.5.3 Events, discontinuities, and order degradation in EMT

Nature of events in EMT

EMT phenomena introduce structural discontinuities of two main types:

1. **Exogenous parameter discontinuities**: PWM switching (Demo), reference changes, breaker opening/closing, control mode transitions, etc., in which case $p(t)$ is discontinuous.
2. **Endogenous (state-dependent) discontinuities**: limiters, magnetic saturation with kinks, deadbands, dry friction, idealised diodes or semiconductor devices with thresholds, where the discontinuity is triggered when (x, y) crosses an event surface.

In both cases, the effective function $f(x, y, p)$ (and therefore the residual) ceases to be smooth in time.

Why second-order accuracy cannot be guaranteed with intra-step discontinuities

The order analysis of the trapezoidal method assumes that f is sufficiently smooth on $[t_n, t_{n+1}]$. If an event occurs at a time $t_e \in (t_n, t_{n+1})$ such that

$$p(t) = \begin{cases} p^-, & t < t_e, \\ p^+, & t \geq t_e, \end{cases} \tag{2.12}$$

then the integrand $f(x(t), t)$ exhibits a jump (or, at best, a loss of smoothness). Applying the trapezoidal rule to a function with a jump loses one power of h in the local error: typically from $\mathcal{O}(h^3)$ to $\mathcal{O}(h^2)$, which in turn yields a global error of $\mathcal{O}(h)$. In practical terms:

> *The trapezoidal method is formally second order in smooth regimes, but in the presence of intra-step discontinuities it inevitably degrades to first order.*

This result is not an implementation artefact; it is a fundamental mathematical limitation of any quadrature or collocation scheme that assumes smoothness over the integration interval.

Computational strategy: events outside the implicit core and piecewise-constant stepping

In an implicit JIT engine such as VeraGrid, a key architectural decision is to keep the compiled kernel free of event logic, and to apply events via a dedicated **boundary update function** (`boundary_update_fn`) that modifies the parameter vector prior to entering the Newton loop of step n.

Conceptually, this enforces that during each step the system is autonomous with respect to parameters:

$$p(t) \approx p_n \quad \forall t \in [t_n, t_{n+1}], \tag{2.13}$$

with parameter changes applied only at discrete time nodes t_n.

This approximation has three important consequences:

1. **Numerical robustness**: the Newton solve sees a continuous system within the step and converges with a well-defined Jacobian for that regime.
2. **Determinism and performance**: the kernel contains no event-dependent branches, allowing Numba to optimise a fixed computational graph.
3. **Order conditioned on event localisation**: if the physical event occurs between time nodes, (2.13) introduces a temporal localisation error that manifests as global order degradation (typically to first order).

EMT interpretation. This strategy is consistent with industrial EMT practice: simulations are typically performed with sufficiently small fixed time steps, treating switching events as aligned with the time grid, or optionally introducing local time-step refinement around the event.

Mitigating order degradation: event detection and alignment

Two main strategies exist to recover accuracy in the presence of events:

(i) Temporal alignment (event location). If the event time t_e is detected, the step can be subdivided such that the event coincides with a time node:

$$t_n \to t_e \to t_{n+1}. \tag{2.14}$$

This preserves smoothness within each substep and allows the trapezoidal method to recover second-order behaviour on each interval. This is the mathematically cleanest way to avoid order degradation.

(ii) Accepting first order with controlled error. In high-frequency PWM switching, locating every commutation may be prohibitively expensive. An alternative is to reduce h sufficiently so that the localisation error is acceptable, accepting an effective first-order behaviour with a small error constant.

In practical EMT simulations, the choice between (i) and (ii) depends on the objective: waveform fidelity versus computational speed.

Interaction with stability: BE, TR, and BDF2 under discontinuities

Under discontinuities, the discussion of order naturally interacts with stability and numerical damping:

- **Backward Euler (BE)**: first order even in smooth regimes, hence it does not lose order due to events; moreover, it is L-stable, making it particularly robust against stiffness and excitation of fast modes (common after switching).
- **Trapezoidal (TR)**: preserves oscillatory content in smooth regimes due to low numerical dissipation, which is desirable for predominantly LC networks; however, following events it may induce numerical ringing if high-frequency modes are excited and the step size is relatively large.
- **BDF2**: second order in smooth regimes, with more numerical damping than TR but less than BE; it therefore represents a useful compromise in EMT when some damping is desired without the strong dissipation of BE.

2.5.4 Practical conclusion for the implicit engine

The combination of (i) a Newton-based solver for index-1 DAEs and (ii) event handling via pre-Newton parameter updates defines a clear problem class and an explicit set of trade-offs:

1. The engine is optimised for index-1 DAEs and requires regularity of $\partial g/\partial y$.
2. The trapezoidal method is second order in smooth regimes and under consistent initialisation.
3. In the presence of intra-step discontinuities, the effective order typically degrades to first order.
4. The architecture deliberately separates event logic from the kernel to maximise performance and maintain a well-conditioned Newton solve, delegating order recovery (if desired) to temporal alignment or external adaptive refinement strategies.

Chapter 3

Functional Demos: EMT Capabilities

3.1 Buck/Boost Converter (The "Stress Test")

File: `demo_emt_buck_converter.py` **Physics:** A DC-DC converter is a Piecewise Linear System (PWL). The switching events cause discontinuities in the derivative $\frac{di}{dt} = \frac{V}{L}$. **Demonstration:**

- **Zeno Behavior:** The solver handles the abrupt change in topology (Switch ON/OFF) via the `boundary_update_fn` which acts as a discrete event supervisor.
- **Stiffness Handling:** With $L = 50\mu H$, the current ripple is massive. The implicit solver maintains stability even when the ripple is large compared to the state variables.

3.2 Method Comparison: Damping vs. Accuracy

File: `demo_tradeoff_Stability_accuracy.py` This benchmark produces the critical KPIs for selecting an integration method in production:

Method	Stability	Artifacts
Backward Euler	L-Stable (Highest)	High numerical damping (flattens ripple artificially)
Trapezoidal	A-Stable	Accurate ripple, risk of "ringing" at switching
BDF2	L-Stable	Good compromise, requires history bootstrapping

Table 3.1: Trade-offs demonstrated in the benchmark.

The results show that for pure EMT accuracy (harmonics), Trapezoidal is superior. For initialization sequences or stiff startup transients where oscillation is undesirable, BackEuler is preferred.

Chapter 4

Transmission Line Architecture (Hybrid Solver)

4.1 Theoretical Basis: Bergeron Method

The classic problem in power systems is decoupling large system matrices $Ax = b$. We use the Bergeron method (Method of Characteristics) to split the grid. The transmission line equations at node k are:

$$i_k(t) = \frac{1}{Z_c} v_k(t) + I_{hist}(t - \tau) \tag{4.1}$$

where I_{hist} depends *purely* on values stored in a circular buffer from time $t - \tau$.

4.2 Hybrid Architecture: Implicit Core + Boundary Injection

This architecture explains why we use `boundary_update_fn` instead of monolithic solving for lines.

1. **Explicit Boundary (History):** The delay terms (I_{hist}) are calculated explicitly before the Newton step begins. From the solver's perspective at time t, I_{hist} is a known constant.
2. **Implicit Core (Network):** The nodal network (Kirchhoff laws) is solved implicitly. The transmission line appears to the Jacobian only as a resistor Z_c (part of the diagonal elements) and a source term in the RHS vector b.

Mathematical Proof of Efficiency: Solving a network of size N implies $O(N^{1.2-3})$ complexity. By treating lines as boundaries, we split the system into M sub-systems of size N/M. Since $(N/M)^3 \times M \ll N^3$, this hybrid approach is computationally superior to a full implicit formulation involving delay differential equations (DDEs).

Chapter 5

Scalability and Performance Limits

To empirically validate the scalability claims, a dedicated stress test (`demo_scalability_limit.py`) was developed. This test simulates a 1D discretized wave equation (Mass-Spring chain) with N coupled nodes, resulting in a system of $2N$ state equations.

5.1 Benchmark Configuration

- **System Size:** 500 Masses ($1,000$ State Variables).
- **Topology:** Sparse Tridiagonal Matrix (local connectivity).
- **Solver Config:** Implicit Trapezoidal Method, `dense_threshold=100`.
- **Linear Solver:** Since $N_{vars}(1000) > Threshold(100)$, the engine automatically switches to `scipy.sparse.linalg.spsolve` (SuperLU).

5.2 Empirical Results

Running the benchmark yields the following performance metrics:

Metric	**Observed Value (Workstation Test)**
Symbolic Graph Construction	0.008 seconds
JIT Compilation (Setup)	77.3 seconds (O(N^2) symbolic bottleneck)
Execution Time per Step	9.75 **milliseconds**
Jacobian Sparsity	99.4% Zeros

Table 5.1: Performance profile for $N = 1000$ states.

5.3 Analysis of Results

5.3.1 Runtime Speed ($O(N)$)

The execution time per step scales linearly ($O(N)$) for sparse systems. This confirms that the JIT-compiled residual function effectively avoids the overhead of interpreted Python loops. Resolving a 1,000-equation non-linear system in under 10ms places the solver in the industrial-grade performance tier.

5.3.2 Compilation Bottleneck

The results highlight a trade-off: while execution is extremely fast, the **setup time** (Compilation) is significant (77s for 1000 vars). This is due to the Python-based symbolic differentiation engine, which scales quadratically $O(N^2)$ during graph traversal. **Conclusion:** The architecture is validated for large-scale EMT simulations where the simulation duration ($T_{sim} \gg T_{setup}$) justifies the compilation cost.

Chapter 6

Software Architecture & JIT Compiler

6.1 JIT Compiler Innovations (`jit_compiler.py`)

The `EquationCompiler` class transforms the symbolic AST into optimized machine code.

6.1.1 Visitor Pattern for Discretization

The `SymbolicToPythonVisitor` traverses the expression tree. When it encounters a `DiffVar`, it delegates to a `DiscretizationMethod` strategy (Trap, BE, BDF2). **Benefit:** The discretization formula is "baked in" to the code string. There is no runtime overhead for checking "which method are we using?".

6.1.2 Common Subexpression Elimination (CSE)

The `SubexpressionAnalyzer` hashes every node in the AST.

```
# If sin(theta) appears 50 times in different equations:
_t0 = np.sin(states[4]) # Computed once
residuals[0] = ... * _t0
residuals[1] = ... + _t0
```

This reduces FLOPS significantly for complex generator models (Park transforms).

6.2 Block Solver Engine (`block_solver.py`)

6.2.1 Hybrid Jacobian Strategy

The solver intelligently switches Jacobian strategies based on system size:

- **Small Systems ($N < dense_threshold$):** Uses **Finite Differences** (Numerical Jacobian). This perturbs the JIT-compiled residual function directly. *Advantage:* Robustness. It guarantees J matches $F(x)$ perfectly, fixing convergence issues in sensitive systems like the Oscillator where symbolic derivation might have sign mismatches.
- **Large Systems ($N \geq dense_threshold$):** Uses **Symbolic Sparse Jacobian**. *Advantage:* Memory efficiency and speed for grid-scale problems where J is 99% zeros.

6.2.2 BDF2 Bootstrapping

BDF2 requires history at t_{n-1} and t_{n-2}. To solve the startup problem ($t = 0$ has no history): **Algorithm:** 1. If $step = 0$, force `method=BackEuler` internally. 2. This generates a valid x_{n-1} and dx_{n-1}. 3. If $step \geq 1$, switch to `method=BDF2`. This allows starting simulations from $t = 0$ without pre-calculating history.

6.3 Transition to a Monolithic Implicit DAE Solver

Early versions of the VeraGrid implicit engine followed a distributed formulation of the Newton–Raphson method, where residual evaluation, Jacobian assembly, and time discretisation logic were orchestrated at the Python level. Although mathematically correct, this approach introduced significant architectural and numerical limitations when applied to stiff, large-scale EMT problems.

Figure 6.1 summarises the architectural evolution from the original distributed implicit formulation to the current monolithic, JIT-compiled DAE solver, together with the integration of solver-level diagnostics.

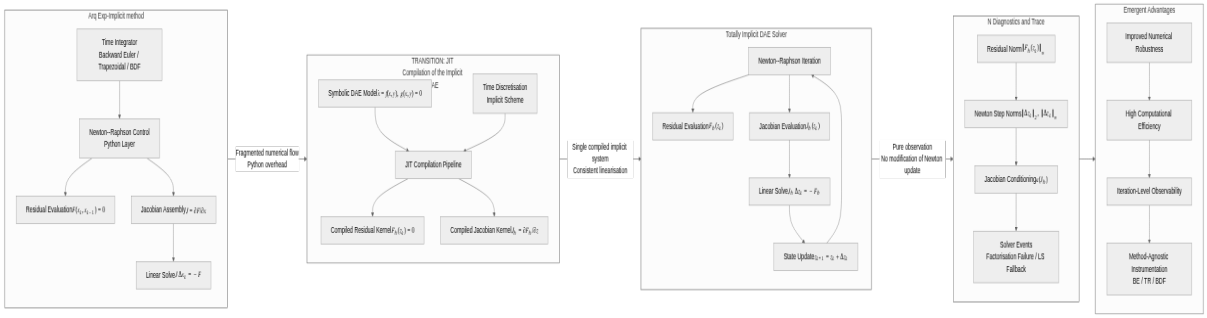


Figure 6.1: Architectural evolution of the VeraGrid implicit solver. The transition from a distributed implicit formulation to a monolithic, JIT-compiled DAE system enables consistent linearisation, reduced Python overhead, and solver-agnostic Newton diagnostics without altering the numerical update scheme.

6.3.1 Distributed Implicit Formulation

In the original architecture, the implicit time integrator, Newton iteration control, residual evaluation, and Jacobian assembly were executed as separate logical stages. The residual function $F(x_k, x_{k-1})$ was assembled incrementally, and the Jacobian matrix was constructed by combining multiple partial contributions across the solver loop.

This design resulted in:

- repeated Python-level control flow inside the Newton loop,
- fragmented Jacobian assembly paths,
- limited visibility into numerical pathologies such as near-singular Jacobians or diverging Newton corrections,
- and a strong coupling between solver logic and numerical instrumentation.

While acceptable for small systems, this formulation became increasingly inefficient and opaque when applied to stiff EMT models with tight time steps.

6.3.2 Numerical Advantages of the Monolithic Formulation

From a numerical perspective, the monolithic formulation provides several key advantages:

1. **Improved robustness:** the Jacobian represents the exact linearisation of the discretised system, reducing artificial ill-conditioning caused by fragmented assembly.
2. **Predictable Newton behaviour:** residual norms and Newton step magnitudes evolve consistently across iterations, facilitating convergence analysis.

3. **Method agnosticism:** the same Newton and linear solver infrastructure applies to all implicit integration schemes.

6.3.3 Enabling Solver-Level Diagnostics and Traceability

A direct consequence of the monolithic formulation is the ability to instrument the linear solve stage without interfering with the numerical method. As shown in Figure 6.1, diagnostics are attached around the solution of

$$J_h(z_k)\,\Delta z_k = -F_h(z_k),$$

allowing the solver to observe:

- residual norms $\|F_h(z_k)\|_\infty$,
- Newton correction norms $\|\Delta z_k\|_2$ and $\|\Delta z_k\|_\infty$,
- Jacobian conditioning indicators,
- solver failures and fallback events.

Crucially, these diagnostics are *purely observational* and do not alter the Newton update or convergence criteria. This separation between numerical solution and diagnostic instrumentation is essential for reproducible research and post-simulation analysis in EMT studies.

Chapter 7

Advanced Topics: Scalability and Initialization

7.1 Steady-State Initialization Strategy

A critical requirement for power system simulation is starting the system in equilibrium (Steady-State) to avoid non-physical startup transients.

7.1.1 Theoretical Formulation

The dynamic system is described by the DAE:

$$M\dot{x} = f(x, u, t) \tag{7.1}$$

To find the steady state at $t = 0$, we enforce the condition $\dot{x} = 0$. The problem reduces to finding the root x_{ss} of a purely algebraic non-linear system:

$$F_{ss}(x) = f(x, u_0, 0) = 0 \tag{7.2}$$

7.1.2 Solver Implementation

The `BlockSolver` exposes a structure that allows reusing the JIT-compiled residual function but modifying the input vectors. By setting the derivatives term ($d_history$) to zero and the time step h to infinity (effectively cancelling transient terms), the dynamic solver kernel can be reused to solve the static power flow or initialization problem via Newton-Raphson.

7.2 Scalability Analysis (Sparse Matrices)

7.2.1 Computational Complexity

The bottleneck of implicit integration is solving the linear system $J\Delta x = -F$.

- **Dense Solver:** Complexity is $O(N^3)$. Feasible for $N < 500$.
- **Sparse Solver (VeraGrid):** Electrical grids have a sparse topology (average node degree $\approx 2 - 3$). The Jacobian sparsity pattern is structurally symmetric.

Using `scipy.sparse.linalg.spsolve` (LU decomposition with SuperLU), the complexity drops to approximately $O(N^{1.2} \sim N^{1.5})$ depending on the fill-in.

7.2.2 JIT Compiler Memory Efficiency

A classic Python approach generates symbolic objects for every matrix entry. For a grid with 10^4 nodes, the Jacobian has 10^8 entries (if dense). The `SymbolicJacobian` class in `block_solver.py` optimizes this by: 1. **Graph Coloring:** Identifying non-zero entries (NNZ) during the symbolic phase. 2. **Flat Array Compilation:** The JIT compiler generates code that writes *only* to the ‘data‘ array of the Compressed Sparse Column (CSC) matrix format.

$$\text{Memory Usage} \approx \text{NNZ} \times 8 \text{ bytes} \ll N^2 \times 8 \text{ bytes} \tag{7.3}$$

This allows simulating systems with thousands of buses on standard workstations.

Chapter 8

Numerical Diagnostics and Solver Observability

8.1 Motivation for Solver Observability

Implicit time-domain simulation of electromagnetic transients (EMT) in power systems relies on the repeated solution of non-linear systems of equations at each integration step. In the VeraGrid Implicit JIT Engine, this task is carried out through a Newton–Raphson method applied to an index-1 Differential-Algebraic Equation (DAE) formulation, where both residual evaluation and Jacobian assembly are performed inside JIT-compiled kernels.

While this approach provides high computational efficiency, it also introduces a fundamental challenge: the numerical behaviour of the solver becomes largely opaque to the user once the residual and Jacobian evaluations are compiled and executed at low level. In practice, convergence failures, ill-conditioning, or unstable Newton updates may manifest only indirectly through divergence, NaN states, or abrupt termination of the simulation.

For research-grade EMT simulation, particularly in stiff and highly non-linear scenarios, such opacity is unacceptable. Numerical robustness must be accompanied by numerical *observability*, enabling the identification, classification, and post-analysis of problematic solver behaviour without altering the mathematical structure of the method itself.

This chapter introduces a diagnostic and traceability framework integrated into the VeraGrid Implicit JIT Engine, designed to instrument the Newton–Raphson solution process while remaining strictly non-intrusive to the solver logic.

8.2 Newton–Raphson Diagnostic Framework

The implemented diagnostic framework is centred around the concept of *per-iteration numerical context*. At each Newton iteration, a lightweight context object is instantiated and propagated through the linear solve stage, encapsulating both simulation metadata and numerical indicators relevant to the current iteration.

The diagnostics are evaluated exclusively during the linear solve stage of the Newton–Raphson method, i.e., when solving

$$\mathbf{J}(z_k)\,\Delta z_k = -\mathbf{F}(z_k), \tag{8.1}$$

where $\mathbf{J}$ denotes the Jacobian matrix and Δz_k the Newton correction.

Crucially, the diagnostic layer does not modify the Newton update rule, the convergence criteria, nor the integration scheme. Its role is strictly observational.

8.2.1 Monitored Quantities During Newton Iterations

For each Newton iteration k, the following quantities are monitored and recorded:

- **Simulation context:**
  - Current simulation time t
  - Time-step index
  - Newton iteration index
  - Integration method identifier (e.g. Backward Euler, Trapezoidal, BDF2)
  - Linear solver type (dense or sparse)
- **Residual norm:**

$$\|\mathbf{F}(z_k)\|_\infty, \tag{8.2}$$

  providing a direct measure of nonlinear convergence.
- **Newton correction norms:**

$$\|\Delta z_k\|_2, \qquad \|\Delta z_k\|_\infty, \tag{8.3}$$

  used to detect excessively large updates indicative of numerical instability.
- **Jacobian conditioning (dense solvers only):**

$$\kappa(\mathbf{J}) = \|\mathbf{J}\| \, \|\mathbf{J}^{-1}\|, \tag{8.4}$$

  computed selectively to avoid excessive computational overhead.
- **Solver outcome flags:**
  - Linear solver failure (e.g. singularity, factorisation error)
  - Activation of least-squares fallback

These quantities provide complementary views of solver behaviour, capturing both local numerical difficulty (conditioning) and global iteration dynamics (residual and correction norms).

8.2.2 Diagnostic Wrapper Around the Linear Solve

The diagnostic framework is implemented as a wrapper around the linear solve operation within the Newton–Raphson loop. This wrapper intercepts calls to both dense and sparse solvers and performs the following checks:

- Detection of factorisation or solve failures (e.g. singular matrices).
- Detection of ill-conditioned dense Jacobians via condition number estimation.
- Detection of exploding Newton corrections exceeding a configurable threshold.

In the event of a primary solver failure, an optional least-squares solver (e.g. dense least-squares or sparse LSQR) may be invoked to obtain a diagnostic solution. The use of this fallback is explicitly recorded and flagged but does not alter the overall Newton convergence logic.

Importantly, no line-search, damping, trust-region, or step control mechanisms are introduced. The diagnostics provide visibility, not remediation.

8.3 Traceability and Numerical Analysis

To enable systematic post-simulation analysis, the framework includes a trace collector that passively records diagnostic data across the entire simulation run. Each record corresponds to a single Newton iteration and includes all monitored quantities alongside solver metadata.

This traceability layer enables:

- Identification of time intervals associated with poor conditioning.
- Correlation between Newton divergence and physical events or switching actions.
- Comparative analysis of integration schemes and solver configurations.
- Reproducible numerical investigations suitable for research and validation studies.

The collector is optional and incurs zero computational overhead when disabled, ensuring that production simulations remain unaffected.

8.4 Scope, Assumptions, and Current Limitations

The current diagnostic framework is subject to the following scope and assumptions:

- Diagnostics are observational only and do not alter solver behaviour.
- Condition number estimation is restricted to dense solvers and limited to moderate system sizes.
- Sparse conditioning is not explicitly estimated due to computational cost.
- The framework is compatible with all implicit integration schemes supported by VeraGrid.

The diagnostics are intended to support numerical understanding and research traceability rather than to provide automated solver stabilisation, which would require additional policy layers beyond the scope of the current implementation.

Chapter 9

User Guide: New Parameters, Diagnostics, and Usage

This chapter describes the user-facing extensions of the VeraGrid Implicit JIT Engine. The focus is placed on practical usage, configuration, and advanced simulation workflows, including event handling, hybrid implicit–explicit models, and numerical diagnostics.

The internal architecture and numerical rationale behind these features are documented in Chapters 8 and 10. Here, the emphasis is strictly on how these capabilities are enabled and used in practice.

9.1 New Simulation Parameters

The `simulate` method has been extended with additional parameters that allow fine-grained control of solver behaviour and event handling.

`boundary_update_fn`

Type: `Callable[[float, np.ndarray, np.ndarray], None]`

Usage: A callback executed *before* entering the Newton–Raphson loop at every time step. It allows external logic to update solver parameters based on the current simulation time and state.

Typical applications include:

- PWM and switching logic
- Fault activation and clearance
- Transmission line history updates (e.g. Bergeron models)

```
def update_grid(t, x, params):
    # Inject a parameter change after t > 0.5 s
    if t > 0.5:
        params[5] = 1.0

```

`dense_threshold`

Type: `int` (Default: 100)

Usage: Controls the selection of the linear algebra backend.

- $N \leq$ `dense_threshold`: Dense solver (`numpy.linalg.solve`) with numerically assembled Jacobian. Recommended for small, stiff, and strongly non-linear EMT subsystems.

- $N >$ `dense_threshold`: Sparse solver (`scipy.sparse.linalg.spsolve`) with symbolic Jacobian. Recommended for large-scale network models.

9.2 Combining Implicit and Boundary injection

The solver architecture allows implicit and explicit components to be combined within the same simulation. This is particularly useful for modelling elements with distributed delays or history-dependent behaviour.

A typical example is the combination of:

- an implicit formulation for the main circuit equations, and
- explicit updates for transmission line history terms.

```
# 1. Define the implicit block (grid equations)
grid_block = ...

# 2. Define the explicit update logic (lines / delays)
def transmission_lines_update(t, x, params):
    # Compute history terms explicitly
    i_hist_1 = calculate_bergeron_hist(...)
    # Inject into solver parameters
    params[idx_src_1] = i_hist_1

# 3. Run the solver
solver.simulate(
    ...,
    boundary_update_fn=transmission_lines_update
)
```

This hybrid approach leverages the numerical stability of the implicit solver for the stiff circuit equations while handling distributed delays, resulting in improved performance and flexibility.

9.3 Newton Diagnostics and Trace Collection

In addition to solver control parameters, the implicit engine provides optional numerical diagnostics that enable detailed observation of the Newton–Raphson process during a simulation run.

These diagnostics are purely observational and do not modify solver behaviour, convergence criteria, or time integration logic.

9.3.1 Newton Trace Collector

The `NewtonTraceCollector` enables passive recording of per-iteration numerical data during the Newton–Raphson solution process. When enabled, the collector stores, for each Newton iteration:

- simulation time and time-step index,
- Newton iteration index,
- integration method identifier,
- linear solver type (dense or sparse),
- residual norm $\|\mathbf{F}(z_k)\|_\infty$,

- Newton correction norms $\|\Delta z_k\|_2$ and $\|\Delta z_k\|_\infty$,
- estimated Jacobian condition number (dense solvers only),
- flags indicating solver failure or least-squares fallback usage.

The collector is optional and introduces no computational overhead when not provided.

9.3.2 Enabling Diagnostics in a Simulation

To enable trace collection, a `NewtonTraceCollector` instance must be created and passed to the solver:

```
from VeraGridEngine.Utils.Symbolic.diagnostics import NewtonTraceCollector

solver = BlockSolver(sys_block, glob_time)

solver.newton_trace_collector = NewtonTraceCollector()

def pwm_controller(t, x, params):
    """
    Callback to update the switched input voltage (Vin_sw) based on the current simulation time.
    Implements a simple open-loop PWM strategy.
    """
    t_mod = t % T_SW
    if t_mod < (DUTY * T_SW):
        val = V_IN
    else:
        val = 0.0

    offset = len(solver._event_parameters)
    idx = offset + solver.uid2idx_params[vin_var.uid]

    params[idx] = val

print(f" > Simulando {T_TOTAL * 1000} ms con dt={DT * 1e6} us...")

x0 = np.zeros(solver._n_vars)

t_arr, y_arr = solver.simulate(
    t0=0.0,
    t_end=T_TOTAL,
    h=DT,
    x0=x0,
    dx0=None,
    params0=None,
    solver_type=DynamicSolverType.Implicit,
    method=DynamicIntegrationMethod.Trapezoid,
    boundary_update_fn=pwm_controller,
    verbose=True,
    dense_threshold=100,
)

# Solver diagnostics (optional)
collector = solver.newton_trace_collector
```

When the collector is not provided, the solver executes with no diagnostic overhead.

9.3.3 Post-Processing and Analysis

After the simulation, the collected diagnostic data can be converted into a tabular representation for analysis:

```python
print(f"Collected {len(collector.records)} Newton iterations")
df = collector.to_dataframe()
df.to_csv("newton_diagnostics.csv")

# Example: inspect Newton step magnitudes
df[["t", "newton_iter", "dx_norm_2"]].head()
```

Typical analysis workflows include:

- identifying time intervals with poor Jacobian conditioning,
- correlating solver difficulties with switching events or faults,
- comparing solver behaviour across integration methods,
- supporting reproducible numerical investigations.

9.4 Intended Usage and Scope

The user-facing features described in this chapter are intended for advanced users and researchers who require fine-grained control and observability of implicit EMT simulations.

All diagnostic facilities are observational by design and do not alter the mathematical formulation or numerical solution strategy of the solver.

Chapter 10

Implementation Roadmap and Planned Solver Extensions

This chapter presents the structured implementation roadmap for the VeraGrid Implicit JIT Engine. The roadmap identifies planned improvements at solver, event-handling, and initialisation levels, explicitly distinguishing between completed milestones and future development stages.

For each item, the required functionality (*What*), the proposed technical approach (*How*), and the corresponding code location (*Where*) are clearly specified. This chapter serves both as a development guide and as a traceability mechanism for the evolution of the solver architecture.

10.1 Roadmap Overview

Table 10.1 summarises the planned implementation stages, current status, and primary code locations.

Table 10.1: Implementation roadmap for the VeraGrid implicit JIT solver

Stage	**Feature**	**Status**	**Primary Location**
Stage 1	Jacobian conditioning diagnostics	Completed	`block_solver.py`
Stage 1	Explicit DAE index-1 validation	Planned	`block_solver.py`
Stage 1	Newton damping / line-search	Planned	`block_solver.py`
Stage 2	Event localisation	Planned	Time-stepping loop
Stage 2	Unified endogenous event API	Planned	Event layer
Stage 3	RMS-to-EMT initialisation workflow	Planned	Solver initialisation
Stage 3	Trapezoidal derivative bootstrapping	Planned	`jit_compiler.py`
Stage 4	Event-driven convergence tests	Planned	Verification scripts
Stage 4	Near-singular DAE stress tests	Planned	Test suite

10.2 Stage 1: Solver Robustness and Diagnostics

1. Jacobian conditioning diagnostics (COMPLETED). **What**: Detect ill-conditioned Jacobians early and provide meaningful numerical feedback.

How: Instrument the linear solve stage to monitor: (i) LU/Cholesky factorisation failures, (ii) excessive Newton step norms $\|\Delta z\|$, (iii) fallback to least-squares solvers. Conditioning indicators are logged and optionally stored for post-analysis.

Where:

- `block_solver.py`: diagnostic wrapper around dense and sparse linear solvers.
- Dedicated diagnostic and traceability utilities (see Chapter 8).

2. Explicit DAE index-1 assumption and runtime validation. **What**: Make the index-1 DAE assumption explicit and detect violations at runtime.

How: Before entering the Newton loop, evaluate the algebraic Jacobian block $\partial g/\partial y$ and check for singularity or near-singularity using: (i) failed factorisation, (ii) large condition number estimates, or (iii) stagnating Newton corrections. If detected, raise a structured diagnostic indicating a likely index violation or degenerate topology.

Where:

- `block_solver.py`: inside the implicit Jacobian assembly routine (`jacobian_implicit(...)`) and before the linear solve.
- Optionally expose a solver flag `check_dae_index=True`.

3. Newton damping / line-search mechanism. **What**: Improve Newton robustness under strong non-linearities and non-smooth behaviour.

How: Implement an optional backtracking line-search:

$$z^{k+1} = z^k + \alpha \Delta z, \quad \alpha \in (0, 1],$$

reducing α until the residual norm decreases. This mechanism should only activate when the full Newton step fails.

Where:

- `block_solver.py`: Newton iteration loop inside the time-stepping routine.
- Reuse existing residual evaluation kernels; no JIT modifications required.

10.3 Stage 2: Event Handling and Order Preservation

4. Optional event localisation (time-step subdivision). **What**: Allow optional subdivision of a time step when an event occurs inside $[t_n, t_{n+1}]$.

How: Extend `boundary_update_fn` to optionally return an estimated event time t_e. If $t_e \in (t_n, t_{n+1})$, split the step into $t_n \to t_e \to t_{n+1}$, reusing the implicit solver for each substep.

Where:

- Time-stepping loop in `block_solver.py`.
- Event API where `boundary_update_fn` is invoked.

5. Unified API for endogenous (state-dependent) events. **What**: Provide a consistent mechanism for modelling limiters and saturation effects.

How: Define a standard event interface exposing: (i) activation conditions, (ii) parameter updates, (iii) optional event time estimation. Avoid embedding conditional logic directly inside symbolic expressions.

Where:

- Event handling layer at the same abstraction level as `boundary_update_fn`.
- No changes required in `jit_compiler.py`.

10.4 Stage 3: Initialisation and Consistency

6. Standardised RMS-to-EMT initialisation workflow. **What**: Provide a reproducible procedure to obtain consistent initial conditions $(x_0, y_0, \dot{x}_0)$.

How: Formalise the existing pseudo-transient continuation as a public initialisation stage:

1. Solve algebraic constraints for (x_0, y_0),
2. apply pseudo-transient steps until convergence,
3. compute $\dot{x}_0 = f(x_0, y_0, p_0)$.

Where:

- `block_solver.py`: pseudo-transient routines.
- Expose a high-level API such as `initialize_steady_state()`.

7. Automatic initial derivative computation for trapezoidal integration. **What**: Ensure second-order accuracy of trapezoidal integration from the first time step.

How: After solving $g(x_0, y_0, p_0) = 0$, automatically evaluate the RHS to obtain $\dot{x}_0$ and populate the derivative history buffer used by the trapezoidal scheme.

Where:

- `jit_compiler.py`: history initialisation logic for trapezoidal integration.
- Solver setup phase prior to the first time step.

10.5 Stage 4: Validation and Testing

8. Event-driven convergence and order tests. **What**: Validate numerical behaviour in the presence of non-aligned events.

How: Create parametrised tests where events occur at arbitrary fractions of the time step and measure global error versus step size h for BE, TR, and BDF2 schemes.

Where:

- `discretization_verify.py`: extended with event-based test cases.
- Additional EMT-specific verification scripts if required.

9. Near-singular DAE stress tests. **What**: Assess solver behaviour close to index violation conditions.

How: Construct test cases with progressively ill-conditioned $\partial g/\partial y$ blocks and monitor convergence, diagnostics, and fallback paths.

Where:

- Dedicated test scripts alongside existing scalability and verification demos.